

Plugging in the lab's production models, biomass, robust water, and non-linear learners

Salil Gujar

M.Tech CSE Entry No. 2025MCS2106

July 2026

Where we are, and what this tackled

Done so far

- A 4-class base map of India at 10 m with a living hierarchy: split, add, rule-split, merge, retrain a node, all as crisp Earth-Engine tiles.
- A git-backed zoo of model and dataset cards, save and resume, GeoTIFF export, per-fortnight water.

This round

- Plug the lab's IndiaSAT models into the framework as usable, carded classifiers.
- Add non-linear learners: Random Forest on Alpha Earth, XGBoost on Tessera.
- Understand the biomass data collection, and what it would take to use it.
- Segment the mining class; make water spatially and temporally robust.
- Implement STACD provenance for every output; measure Tessera against Alpha Earth.

Plugging in the lab's production models

Use the IndiaSAT models shared with us: train and store them, and list them in the zoo so a user can pick one to split a class.

- Two models: a pan-India *tree against crop* classifier, and a per-region *farm, plantation, scrubland* classifier.
- We found both train a Random Forest *and* classify entirely inside Earth Engine, so they render server-side as tiles, with nothing downloaded and no re-implementation on our side.
- Their training data is readable from our project, so we reproduce them faithfully (trained on the fly from the stored asset) rather than copying weights.
- Each is now a card in the zoo, applied from there.

The two models, and how they plug in

- Tree vs crop, pan-India: features are a Sentinel-1 radar time series (23 sixteen-day steps, two polarisations), trained on about seventy thousand labelled points.
- Farm / plantation / scrubland, per agro-ecological region: features are the same Alpha Earth embedding we already use, trained per region on about 1.5 million points.
- Both are a new card topology, `ee_rfx`, storing the recipe (asset, features, classes), not a file, since the model is re-trained in Earth Engine on demand.
- They carry their own class scheme, so applying one *rewrites the hierarchy*: greenery becomes the model's classes and the rest of the base map (built-up, water, barren) is kept. The tree and the map both follow the model.

Non-linear learners, and where they run

- The aim was Random Forest on Earth Engine, the model that usually works, and XGBoost on Tessera. XGBoost on Tessera was almost free; Random Forest on Alpha Earth was the real work.
- A Random Forest is not band math, so inference is now *algorithm-aware*: a non-linear Alpha Earth split falls back to the point-grid render (the path Tessera already uses), while linear models keep the crisp tile map.
- This also unlocks biomass, a Random Forest regressor on the same Alpha Earth features.

Biomass from GEDI: the data collection, and findings

A study of the biomass scripts we were given, to understand the data collection and how it would fit. It is not surfaced in the interface.

- The collection samples GEDI, a lidar that measures above-ground biomass, and pairs each shot with the same Alpha Earth embedding we classify on, plus slope.
- Finding: biomass is a regression target on our own feature space, so it slots in as a Random Forest regressor on the same features, with no new pipeline.
- We reproduced the collection and reached the expected ballpark; it stays out of the interface until we decide how biomass should appear in the LULC.

Segmenting the mining class

- The mining class is per-pixel, so it shows as scattered pixels; the ask was mining as discrete objects.
- This is *not* the object-detection model intended for this: that needs imagery, masks, and GPU we have not wired up. As an interim that reuses the existing tile path, we vectorise the mining prediction in Earth Engine: de-speckle, close holes, trace polygons, drop tiny blobs.
- Result: clean mining polygons with a per-object area, downloadable, instead of pixel confetti, useful now. The learned object-detection model is the real next step.

Water, step one: robust across bodies and years

Two steps: first get water against non-water working and see the accuracy, then decide how to fold it into the LULC. This is step one.

- We hold out whole water bodies (spatial), whole years (temporal), and the combination (the honest worst case). The polygons already carry a water-body identity and a date.
- Within water bodies the task is strong and stable: the combined spatial-and-temporal hold-out scores about 0.98, with a small year-to-year spread, so no fluke year.
- So the classifier is solid enough within water bodies to build on.

Water, step one: works anywhere, and counting fortnights

- The catch: the non-water examples all come from around water bodies, so the model over-calls water on dry land. We augment non-water with barren, built-up, and greenery pixels across seasons.
- The effect: non-water precision rises from about 0.72 to 0.99, so it stops painting dry land as water. This works-anywhere model is now deployed.
- We also run the model over a year of fortnights and count, per pixel, how many fortnights held water, so perennial bodies score high and monsoon-only ponds low. This is the seasonal-water signal the LULC would use, deferred to step two.

Acacia: measuring spatial and temporal robustness

Acacia against non-acacia is a hard species split (two similar trees). The point here is the honest measurement, not a headline number: crowns now carry their source region, so we can hold out a whole region and whole years at once.

What is held out	Accuracy
Unseen years only (temporal)	0.749
An unseen region only (spatial)	0.695
An unseen region <i>and</i> unseen years	0.679

The ordering is what it should be: a whole region is harder than random polygons, and region-and-year together is hardest. That gap is the value, since it shows how much a single-site, single-year acacia number would over-promise. Reassuringly, across the two test years the accuracy is stable (spread ~ 0.002), so there is no fluke year: the split travels consistently, just modestly, because it is a hard species distinction.

How long does Tessera take, against Alpha Earth

Measured live over one small site, to advise which pipeline to reach for.

Stage	Tessera	Alpha Earth
Download	~29 s / tile (151 MB)	none, server-side
Sample	31 s	4 s
Train	<1 s	<1 s
Classify	40 s, point grid	8 s, tiles
Total	~72 s	~12 s

Alpha Earth is server-side round-trips only. Tessera pays a large per-tile download first, then samples and classifies locally. So Alpha Earth wins on first touch; Tessera is worth it only when its local features are needed and the download is already paid.

STACD provenance for every output

STACD provenance was implemented for every classified output, following the paper's five classes.

- Each output emits a STAC Item for the raster (box, geometry, class legend, assets) plus the STACD graph: a node per input dataset and algorithm, one instance per live model, rule, and merge, and the output instance.
- The class hierarchy and the ordered operations that built it are embedded as the producing algorithm's input set, reusing the zoo cards, so there is no new source of truth and no Earth-Engine run.
- We emit the metadata half; the paper's Airflow runtime is the next layer. The record lines up with the drone and bioacoustics outputs, so it can be compared across the projects.

Thank you